



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Контрольная работа № 1

по курсу «Языки и методы программирования»

«Мониторинг директории. MQTT. Многопоточность»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Архитектура приложения	1
Класс FileMonitorTask	1
Класс FileParserTask	4
Класс SharedMessageQueue	6
Класс MqttPublisherTask	8
Класс MqttListenerTask	10
Класс MainWindow	12
Класс App	14
Сборка и запуск	16
Демонстрация работы	16
Вывод	17

Цель работы

Разработать многопоточное Java-приложение, которое:

- отслеживает появление новых файлов в заданной директории с помощью системных вызовов;
- читает файлы формата ключ:значение, сохраняя порядок по времени создания;
- накапливает записи в очереди с ограниченной ёмкостью;
- публикует накопленные записи в MQTT-брокер;
- отображает состояние очереди файлов, очереди отправки и полученные из MQTT сообщения в графическом интерфейсе.

Архитектура приложения

Приложение построено по конвейерной схеме (pipeline) из четырёх потоков, связанных двумя разделяемыми очередями:

FileMonitorTask (наблюдение) -> BlockingQueue -> FileParserTask (парсинг) ->
SharedMessageQueue (ёмкость 5, wait/notify) -> MqttPublisherTask
(публикация) -> [MQTT Broker] (Mosquitto :1883) -> MqttListenerTask
(подписка) -> MainWindow (GUI)

Управляющий класс App создаёт все компоненты и запускает четыре фоновых потока. GUI обновляется через колбэки, которые используют `SwingUtilities.invokeLater()` для безопасного доступа к Swing-компонентам из рабочих потоков.

Класс FileMonitorTask

Отвечает за отслеживание появления новых файлов в директории `target-dir/`.

- Использует `java.nio.file.WatchService` — на Linux это системный вызов `inotify`, что обеспечивает мгновенную реакцию на события, а не периодический опрос (premium-задача из ТЗ).
- После получения события `ENTRY_CREATE` ожидает 300 мс, чтобы файл успел завершить запись.

- Собирает все события из одного WatchKey в пакет и сортирует файлы по времени создания (creationTime / birth date) — это гарантирует правильный порядок обработки.
- Добавляет файлы в общую BlockingQueue<File> и уведомляет GUI о новом файле.

FileMonitorTask.java

```
package com.example;

import java.io.File;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.concurrent.BlockingQueue;
import java.util.function.Consumer;

public class FileMonitorTask implements Runnable {
    private final Path dir;
    private final BlockingQueue<File> fileQueue;
    private final Consumer<String> onFileAdded;

    public FileMonitorTask(Path dir, BlockingQueue<File> fileQueue,
        Consumer<String> onFileAdded) {
        this.dir = dir;
        this.fileQueue = fileQueue;
        this.onFileAdded = onFileAdded;
    }

    @Override
    public void run() {
        try (WatchService watchService =
            FileSystems.getDefault().newWatchService()) {
            dir.register(watchService,
                StandardWatchEventKinds.ENTRY_CREATE);

            while (!Thread.currentThread().isInterrupted()) {
                WatchKey key = watchService.take();

                List<File> batch = new ArrayList<>();
                for (WatchEvent<?> event : key.pollEvents()) {
                    Path context = (Path) event.context();
                    File newFile = dir.resolve(context).toFile();
                    Thread.sleep(300);
                    batch.add(newFile);
                }
                key.reset();

                batch.sort(Comparator.comparingLong(f -> {
                    try {
                        return Files.readAttributes(f.toPath(),
                            BasicFileAttributes.class)
                            .creationTime().toMillis();
                    } catch (Exception e) {
                        return 0L;
                    }
                }));
            }
        }
    }
}
```

Класс `FileParserTask`

Потребляет файлы из очереди, читает их и разбирает строки формата ключ:значение.

- Блокируется на `fileQueue.take()` до появления нового файла.
- Перед чтением ожидает 1 секунду — гарантирует, что файл полностью записан на диск.
- Каждую строку разделяет по первому двоеточию (`split(":", 2)`) и добавляет пару в `SharedMessageQueue`.
- Между строками делает паузу 500 мс для визуализации пошаговой обработки.
- После обработки удаляет файл (destructive consumption per T3).
- Сообщает GUI об удалении файла из списка.

FileParserTask.java

```
package com.example;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.util.concurrent.BlockingQueue;
import java.util.function.Consumer;

public class FileParserTask implements Runnable {
    private final BlockingQueue<File> fileQueue;
    private final SharedMessageQueue messageQueue;
    private final Consumer<String> onFileRemoved;

    public FileParserTask(BlockingQueue<File> fileQueue,
        SharedMessageQueue messageQueue,
        Consumer<String> onFileRemoved) {
        this.fileQueue = fileQueue;
        this.messageQueue = messageQueue;
        this.onFileRemoved = onFileRemoved;
    }

    @Override
    public void run() {
        try {
            while (!Thread.currentThread().isInterrupted()) {
                File file = fileQueue.take();

                Thread.sleep(1000);

                try (BufferedReader br =
                    new BufferedReader(new FileReader(file))) {
                    String line;
                    while ((line = br.readLine()) != null) {
                        String[] parts = line.split(":", 2);
                        if (parts.length == 2) {
                            messageQueue.addMessage(
                                parts[0].trim(), parts[1].trim());
                            Thread.sleep(500);
                        }
                    }
                }
                file.delete();
                onFileRemoved.accept(file.getName());
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Класс `SharedMessageQueue`

Потокобезопасная очередь сообщений с ограниченной ёмкостью (5), реализованная через низкоуровневый монитор Java (`synchronized`, `wait/notifyAll`).

- `addMessage(key, value)` — добавляет пару, если очередь не заполнена; при достижении ёмкости будит ожидающий поток через `notifyAll()`.
- `takeAllWhenFull()` — блокируется, пока очередь не заполнена; затем извлекает все сообщения разом (batch) и очищает очередь.
- `getSize()` / `getCapacity()` — используются GUI для отображения прогресса.
- После каждого изменения вызывает `guiCallback`, который обновляет `JProgressBar` на экране.

SharedMessageQueue.java

```
package com.example;

import java.util.ArrayList;
import java.util.List;

public class SharedMessageQueue {
    private final int capacity;
    private final List<String> messages = new ArrayList<>();
    private final Runnable guiCallback;

    public SharedMessageQueue(int capacity,
                               Runnable guiCallback) {
        this.capacity = capacity;
        this.guiCallback = guiCallback;
    }

    public synchronized void addMessage(
        String key, String value) {
        if (messages.size() < capacity) {
            messages.add(key + ":" + value);
            guiCallback.run();

            if (messages.size() == capacity) {
                notifyAll();
            }
        }
    }

    public synchronized List<String> takeAllWhenFull()
        throws InterruptedException {
        while (messages.size() < capacity) {
            wait();
        }

        List<String> batch = new ArrayList<>(messages);
        messages.clear();
        guiCallback.run();
        return batch;
    }

    public synchronized int getSize() {
        return messages.size();
    }

    public int getCapacity() {
        return capacity;
    }
}
```


Класс MqttPublisherTask

Публикует сообщения в MQTT-брокер на топик "iu9".

- Подключается к брокеру `tcp://localhost:1883` с помощью Eclipse Paho.
- В цикле вызывает `messageQueue.takeAllWhenFull()` — блокируется до накопления 5 сообщений.
- Перед публикацией делает паузу 1 секунду.
- Публикует каждое сообщение отдельным `MqttMessage` (без сохранения на диске — `MemoryPersistence`).

MqttPublisherTask.java

```
package com.example;

import org.eclipse.paho.client.mqttv3.MqttClient;
import org.eclipse.paho.client.mqttv3.MqttMessage;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
import java.util.List;

public class MqttPublisherTask implements Runnable {
    private final SharedMessageQueue messageQueue;
    private final String brokerUrl;
    private final String topic;

    public MqttPublisherTask(
        SharedMessageQueue messageQueue,
        String brokerUrl, String topic) {
        this.messageQueue = messageQueue;
        this.brokerUrl = brokerUrl;
        this.topic = topic;
    }

    @Override
    public void run() {
        try {
            MqttClient client = new MqttClient(brokerUrl,
                MqttClient.generateClientId(),
                new MemoryPersistence());
            client.connect();

            while (!Thread.currentThread().isInterrupted()) {
                List<String> batch =
                    messageQueue.takeAllWhenFull();

                Thread.sleep(1000);

                for (String msg : batch) {
                    MqttMessage mqttMsg =
                        new MqttMessage(msg.getBytes());
                    client.publish(topic, mqttMsg);
                    System.out.println(
                        "Опубликовано: " + msg);
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Класс MqttListenerTask

Подписывается на тот же топик "iu9" и отображает входящие сообщения в правой колонке GUI.

- Подключается к брокеру и регистрирует MqttCallback.
- В messageArrived делает паузу 1 секунду и передаёт сообщение в GUI через Consumer<String>.
- Поток завершается после подписки, но RaHo продолжает доставлять сообщения через внутренний диспетчерский поток.

MqttListenerTask.java

```
package com.example;

import org.eclipse.paho.client.mqttv3.*;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;
import java.util.function.Consumer;

public class MqttListenerTask implements Runnable {
    private final String brokerUrl;
    private final String topic;
    private final Consumer<String> onMessageReceived;

    public MqttListenerTask(String brokerUrl, String topic,
        Consumer<String> onMessageReceived) {
        this.brokerUrl = brokerUrl;
        this.topic = topic;
        this.onMessageReceived = onMessageReceived;
    }

    @Override
    public void run() {
        try {
            MqttClient client = new MqttClient(brokerUrl,
                MqttClient.generateClientId(),
                new MemoryPersistence());
            client.setCallback(new MqttCallback() {
                public void connectionLost(Throwable cause) {}
                public void messageArrived(
                    String topic, MqttMessage message) {
                    try { Thread.sleep(1000); }
                    catch (InterruptedException e) {
                        Thread.currentThread().interrupt(); }
                    onMessageReceived.accept(
                        new String(message.getPayload()));
                }
                public void deliveryComplete(
                    IMqttDeliveryToken token) {}
            });
            client.connect();
            client.subscribe(topic);
            System.out.println(
                "Слушатель подключен к топику: " + topic);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Класс `MainWindow`

Графический интерфейс на Swing с тремя колонками (`GridLayout(1, 3)`).

- Левая колонка «Очередь файлов» — `JList` с именами файлов, ожидающих обработки.
- Центральная колонка «Очередь на отправку» — `JProgressBar` с подписью `current / max`.
- Правая колонка «Получено из MQTT» — `JList` с сообщениями из брокера.
- Все методы обновления UI обёрнуты в `SwingUtilities.invokeLater()` для потокобезопасности.

MainWindow.java

```

package com.example;

import javax.swing.*.*;
import java.awt.*.*;

public class MainWindow extends JFrame {
    private DefaultListModel<String> fileListModel;
    private JProgressBar queueProgressBar;
    private DefaultListModel<String> mqttListModel;

    public MainWindow() {
        setTitle("File & MQTT Thread Manager");
        setSize(900, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new GridLayout(1, 3, 10, 10));

        fileListModel = new DefaultListModel<>();
        JList<String> fileList =
            new JList<>(fileListModel);
        JPanel col1 = new JPanel(new BorderLayout());
        col1.add(new JLabel("Очередь файлов",
            SwingConstants.CENTER),
            BorderLayout.NORTH);
        col1.add(new JScrollPane(fileList),
            BorderLayout.CENTER);

        queueProgressBar = new JProgressBar(0, 100);
        queueProgressBar.setStringPainted(true);
        JPanel col2 = new JPanel(new BorderLayout());
        col2.add(new JLabel("Очередь на отправку",
            SwingConstants.CENTER),
            BorderLayout.NORTH);
        col2.add(queueProgressBar,
            BorderLayout.CENTER);

        mqttListModel = new DefaultListModel<>();
        JList<String> mqttList =
            new JList<>(mqttListModel);
        JPanel col3 = new JPanel(new BorderLayout());
        col3.add(new JLabel("Получено из MQTT",
            SwingConstants.CENTER),
            BorderLayout.NORTH);
        col3.add(new JScrollPane(mqttList),
            BorderLayout.CENTER);

        add(col1);
        add(col2);
        add(col3);
    }
}

```

Класс App

Точка входа в приложение. Выполняет инициализацию и запуск всех компонентов.

- Устанавливает FlatLaf (FlatLightLaf) в качестве look-and-feel.
- Создаёт директорию target-dir/, если её нет.
- Создаёт LinkedBlockingQueue<File> для передачи файлов от монитора к парсеру.
- Создаёт SharedMessageQueue ёмкостью 5 с колбэком для обновления GUI.
- Создаёт MainWindow и отображает его в EDT.
- Запускает 4 потока: FileMonitorTask, FileParserTask, MqttPublisherTask, MqttListenerTask.

App.java

```
package com.example;

import com.formdev.flatlaf.FlatLightLaf;
import javax.swing.*.*;
import java.io.File;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

public class App {
    public static void main(String[] args) {
        FlatLightLaf.setup();

        String brokerUrl = "tcp://localhost:1883";
        String topic = "iu9";
        int MESSAGE_QUEUE_CAPACITY = 5;
        Path watchDir = Paths.get("target-dir");

        File dir = watchDir.toFile();
        if (!dir.exists())
            dir.mkdirs();

        BlockingQueue<File> fileQueue =
            new LinkedBlockingQueue<>();
        MainWindow window = new MainWindow();

        final SharedMessageQueue[] msgQueueHolder =
            new SharedMessageQueue[1];
        SharedMessageQueue messageQueue =
            new SharedMessageQueue(
                MESSAGE_QUEUE_CAPACITY,
                () -> window.updateMessageQueueUI(
                    msgQueueHolder[0]));
        msgQueueHolder[0] = messageQueue;

        SwingUtilities.invokeLater(() -> {
            window.setVisible(true);
            window.updateMessageQueueUI(messageQueue);
        });

        Thread t1 = new Thread(new FileMonitorTask(
            watchDir, fileQueue,
            window::addFileToList));
        Thread t2 = new Thread(new FileParserTask(
            fileQueue, messageQueue,
            window::removeFileFromList));
        Thread t3 = new Thread(new MqttPublisherTask(
            messageQueue, brokerUrl, topic));
```


Сборка и запуск

Сборка осуществляется через `make`. Для запуска требуется MQTT-брокер Mosquitto в Docker.

Makefile

```
CP = out:lib/flatlaf-3.7.jar:\
    lib/org.eclipse.paho.client.mqttv3-1.2.5.jar

compile:
    mkdir -p out
    javac -d out -cp "$(CP)" src/com/example/*.java

run: compile
    java -cp "$(CP)" com.example.App

run-mqtt:
    docker run --rm -it -p 1883:1883 \
        -v "$(PWD)/mosquitto.conf:/mosquitto/config/mosquitto.conf" \
        eclipse-mosquitto:latest
```

Конфигурация Mosquitto разрешает анонимное подключение на порту 1883:

mosquitto.conf

```
listener 1883
allow_anonymous true
```

Сборка и запуск

```
# 1. Запустить MQTT-брокер
make run-mqtt

# 2. В другом терминале собрать и запустить приложение
make run
```

Демонстрация работы

Для тестирования используется скрипт `a.sh`, копирующий тестовый файл `1.txt` в директорию `target-dir/` 13 раз.

```
1.txt

a:1
b:2
c:3
d:4
```

Каждый файл содержит 4 строки ключ:значение. Приложение отслеживает появление файлов через WatchService, сортирует по времени создания, читает и парсит их. Сообщения накапливаются в SharedMessageQueue (ёмкость 5), после чего публикуются в MQTT-брокер. MqttListenerTask получает их обратно и отображает в правой колонке GUI.

Вывод

В ходе контрольной работы разработано многопоточное Java-приложение с графическим интерфейсом, осуществляющее:

- мониторинг файловой системы через WatchService (inotify) — без polling, с сохранением порядка по времени создания файлов;
- разбор текстовых файлов формата ключ:значение;
- накопление сообщений в ограниченной очереди с использованием низкоуровневых примитивов синхронизации (wait/notify);
- публикацию и подписку на сообщения через MQTT (Eclipse Paho);
- отображение всех стадий обработки в Swing-интерфейсе с тремя панелями.

Архитектура построена по шаблону конвейера (pipeline) с четырьмя потоками и двумя разделяемыми очередями. GUI обновляется асинхронно через колбэки с использованием `SwingUtilities.invokeLater()`, что гарантирует потокобезопасность.

Особенностью реализации является использование системных вызовов ядра для отслеживания файлов (премиальная задача из ТЗ), а также ручная реализация bounded blocking queue на мониторах Java без применения высокоуровневых утилит `java.util.concurrent`.